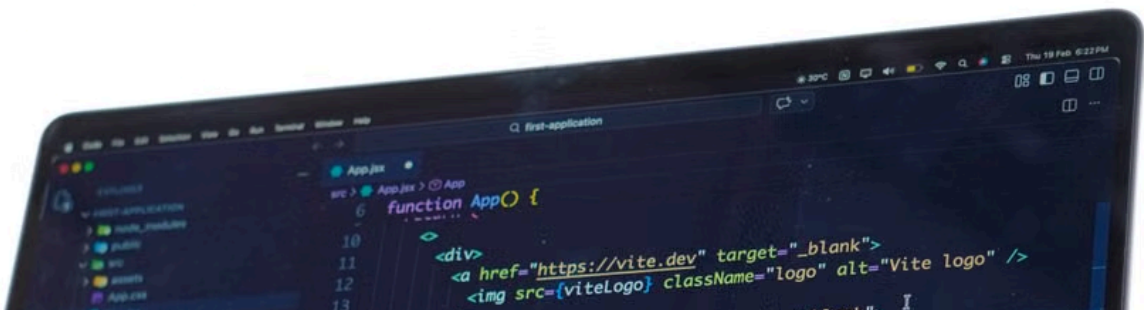




Aplicaciones web 1

Actividad 3

Actividad a desarrollar en grupo. La entrega es individual para que quede el registro.

Infraestructura, dominios y arquitectura web

Continuar con el caso de negocio de la actividad 1 y 2.

Consignas

- **Investigación de dominio real:**
 - Proponer el nombre de dominio deseado para el proyecto.
 - Ingresar a una plataforma proveedora (ej. NIC Argentina, Namecheap, DonWeb), y verificar la disponibilidad del dominio real y documentar el precio anual estimado.
 - Justificar la elección del TLD seleccionado.
Ejemplo: **.com** vs. **.com.ar** vs. **.app**.
- **Diagramación del flujo cliente-servidor (petición HTTP):** Realizar un esquema gráfico explicativo en papel, Canva o Figma que muestre el recorrido completo desde que el usuario ingresa la **URL** en el navegador hasta que la página se renderiza.

Explicar:

- Servidor DNS (Resolución de IP).
- Petición HTTP GET al servidor web (hosting). Incluir capturas de pantalla de la herramienta del navegador DevTools -> Network -> (click sobre un

recurso).

- Respuesta del servidor enviando los archivos o recursos. Incluir capturas de pantalla de la herramienta del navegador DevTools -> Network -> (click sobre un recurso).

Uso obligatorio de inteligencia artificial (IA)

- **Generación de nombres de dominio creativos:** Utilizar la IA para hacer un brainstorming de dominios disponibles y verificar alternativas según disponibilidad y SEO.
"Dame 10 opciones de nombres de dominio cortos, memorables y profesionales para [tipo de negocio en Argentina]. Incluye opciones con terminación .com y .com.ar."
- **Explicación de conceptos de red:** Usar la IA para validar el diagrama cliente-servidor, pidiéndole que explique en términos sencillos el paso a paso de una petición HTTP GET y el rol de un servidor DNS.
- **Tarea del grupo:** Auditar la respuesta de la IA, ajustar los textos según el criterio propio del grupo y **refinar la redacción final.**

Resolver aquí. Descargar en PDF y subir al repositorio.

Investigación del Dominio real:

Namecheap:

✓ sisfogym.com	\$6.79 WITH NEWCOM679 ⓘ ☆	\$11.28/yr Retail \$14.98/yr	🛒 Add to cart
✓ sisfofit.com	\$6.79 WITH NEWCOM679 ⓘ ☆	\$11.28/yr Retail \$14.98/yr	🛒 Add to cart
✓ sisifogymfit.com	\$6.79 WITH NEWCOM679 ⓘ ☆	\$11.28/yr Retail \$14.98/yr	🛒 Add to cart
✓ teamsisifo.com	\$6.79 WITH NEWCOM679 ⓘ ☆	\$11.28/yr Retail \$14.98/yr	🛒 Add to cart

DonWeb:

✓ ¡Tu dominio está disponible!
Incluye un .ONLINE y un .STORE de regalo.

gymsisifo.com.ar

\$17.900 /año

CONTINUAR

🌐 También te recomendamos

gymsisifo.com

\$16.900 /año

CONTINUAR

✓ ¡Tu dominio está disponible!
Incluye un .ONLINE y un .STORE de regalo.

sisifogym.com

\$16.900 /año

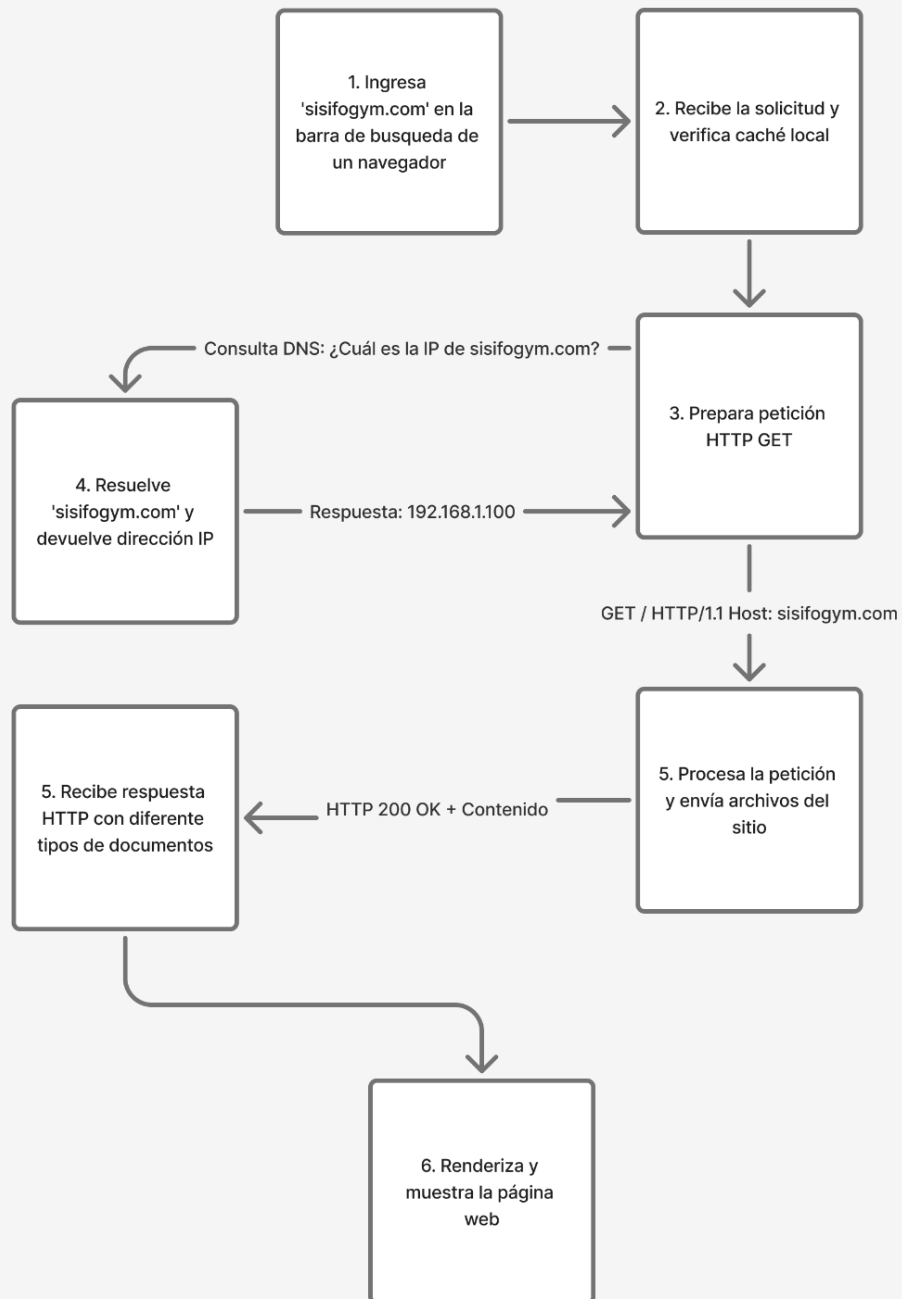
CONTINUAR

Seleccionamos el dominio sisifogym.com porque mantiene de forma fiel e intuitiva la identidad de la marca. Al emplear la extensión más habitual, se facilita la búsqueda directa y es fácil de recordar por parte de los usuarios.

Diagramación del flujo cliente-servidor (petición HTTP):

- 1. El usuario ingresa la URL en su navegador y presiona Enter.**
- 2. El navegador (cliente) consulta al servidor DNS, que transforma el nombre de dominio de la URL en una dirección IP para poder localizar el servidor.**
- 3. El navegador se comunica con el servidor de hosting donde se encuentra alojada la página web.**
- 4. El navegador realiza una petición HTTP mediante el método GET para solicitar la página o el recurso correspondiente.**
- 5. El servidor recibe y procesa la petición GET, busca el recurso solicitado y prepara una respuesta.**
- 6. El servidor devuelve una respuesta HTTP con el recurso solicitado, por ejemplo, el archivo HTML de la página.**
- 7. El navegador analiza el HTML recibido y comprueba si necesita otros recursos, como archivos CSS, JavaScript, imágenes o fuentes. Para obtenerlos, realiza nuevas peticiones GET al servidor.**
- 8. Finalmente, el navegador procesa y reúne todos los recursos recibidos —HTML, CSS, JavaScript, imágenes, etc.— y renderiza la página web, mostrándola al usuario.**

Diagrama de flujo



Capturas de pantalla:

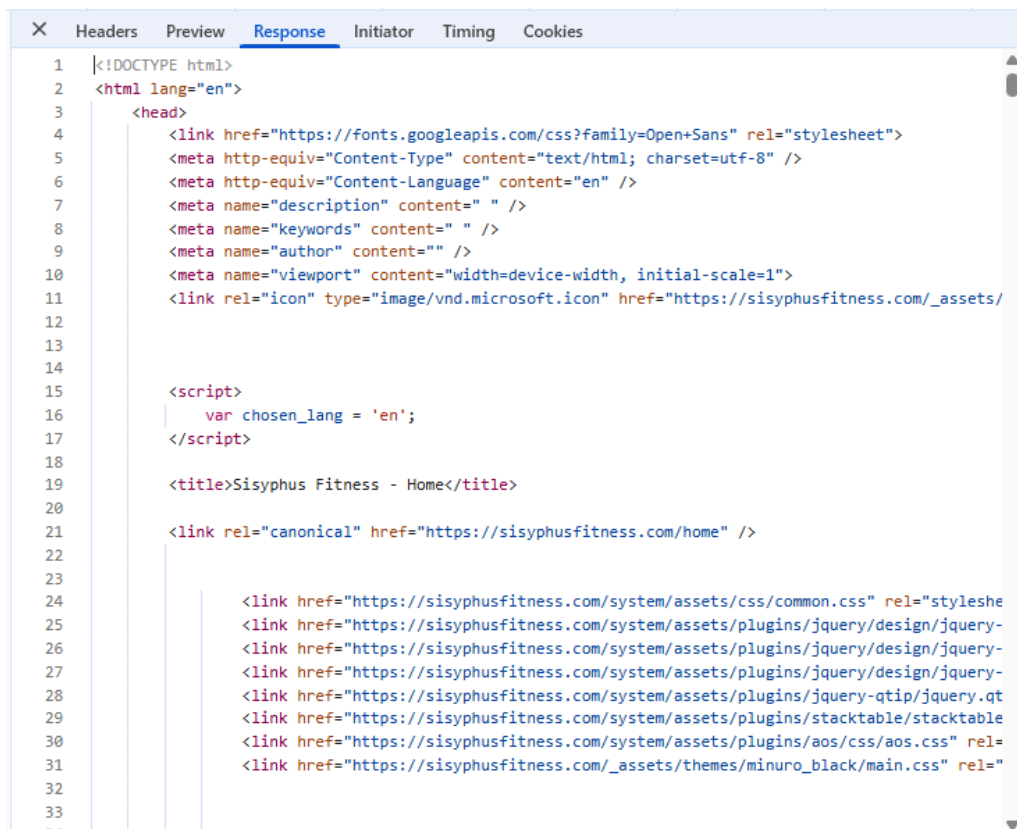
Lista de peticiones:

Name	Status	Type	Initiator	Size	Time
home	200	document	Other	5.7 kB	274 ms
css?family=Open+Sans	200	stylesheet	home:4	(memory ca...	0 ms
common.css	200	stylesheet	home:24	(memory ca...	0 ms
jquery-ui.min.css	200	stylesheet	home:25	(memory ca...	0 ms
jquery-ui.structure.min.css	200	stylesheet	home:26	(memory ca...	0 ms
jquery-ui.theme.css	200	stylesheet	home:27	(memory ca...	0 ms
jquery.qtip.min.css	200	stylesheet	home:28	(memory ca...	0 ms
stacktable.css	200	stylesheet	home:29	(memory ca...	0 ms
aos.css	200	stylesheet	home:30	(memory ca...	0 ms
main.css	200	stylesheet	home:31	(memory ca...	0 ms
jquery.min.js	200	script	home:36	(memory ca...	0 ms
jquery.cookie.js	200	script	home:37	(memory ca...	0 ms
jquery-ui.js	200	script	home:38	(memory ca...	0 ms
jquery.qtip.min.js	200	script	home:39	(memory ca...	0 ms
stacktable.js	200	script	home:40	(memory ca...	0 ms
user_api.js	200	script	home:41	(memory ca...	0 ms
ace.js	200	script	home:42	(memory ca...	0 ms
jquery.sticky.js	200	script	home:43	(memory ca...	0 ms
lazyload.min.js	200	script	home:44	(memory ca...	0 ms
aos.js	200	script	home:45	(memory ca...	0 ms
theme.js	200	script	home:46	(memory ca...	0 ms
parallax.min.js	200	script	home:47	(memory ca...	0 ms
logo_1.png	200	png	home:169	(memory ca...	0 ms
font-awesome.min.css	200	stylesheet	main.css:1	(memory ca...	0 ms
memSYaGs126MiZpBA-UvWbX2vVnXBbObj2O...	200	font	css	(memory ca...	0 ms
fontawesome-webfont.woff?v=4.7.0	200	font	font-awesome.min.css	(memory ca...	0 ms
OpenSans-SemiBold.woff	200	font	main.css	(memory ca...	0 ms

Recurso:Home

▼ General	
Request URL	https://sisyphusfitness.com/home
Request Method	GET
Status Code	● 200 OK
Remote Address	116.203.101.212:443
Referrer Policy	origin

Contenido que muestra de la página:



```
1 |<!DOCTYPE html>
2 |<html lang="en">
3 |   <head>
4 |     <link href="https://fonts.googleapis.com/css?family=Open+Sans" rel="stylesheet">
5 |     <meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
6 |     <meta http-equiv="Content-Language" content="en" />
7 |     <meta name="description" content=" " />
8 |     <meta name="keywords" content=" " />
9 |     <meta name="author" content=" " />
10 |    <meta name="viewport" content="width=device-width, initial-scale=1">
11 |    <link rel="icon" type="image/vnd.microsoft.icon" href="https://sisyphusfitness.com/_assets/
12 |
13 |
14 |
15 |    <script>
16 |      var chosen_lang = 'en';
17 |    </script>
18 |
19 |    <title>Sisyphus Fitness - Home</title>
20 |
21 |    <link rel="canonical" href="https://sisyphusfitness.com/home" />
22 |
23 |
24 |    <link href="https://sisyphusfitness.com/system/assets/css/common.css" rel="styleshe
25 |    <link href="https://sisyphusfitness.com/system/assets/plugins/jquery/design/jquery-
26 |    <link href="https://sisyphusfitness.com/system/assets/plugins/jquery/design/jquery-
27 |    <link href="https://sisyphusfitness.com/system/assets/plugins/jquery/design/jquery-
28 |    <link href="https://sisyphusfitness.com/system/assets/plugins/jquery-qtip/jquery.q
29 |    <link href="https://sisyphusfitness.com/system/assets/plugins/stacktable/stacktable
30 |    <link href="https://sisyphusfitness.com/system/assets/plugins/aos/css/aos.css" rel=
31 |    <link href="https://sisyphusfitness.com/_assets/themes/minuro_black/main.css" rel="
32 |
33 |
```

Prompt 1: Dame 10 opciones de nombres de dominio cortos, memorables y profesionales para sisifogym. Incluye opciones con terminación .com y .com.ar.

Respuesta:

Para **Sisifo Gym**, las opciones combinan fuerza, brevedad y facilidad para recordar, distribuidas entre .com y .com.ar:

Opciones .com (Internacionales y directas)

- **sisifogym.com** — La opción directa y natural de la marca.
- **somosisifo.com** — Genera sentido de comunidad y pertenencia.
- **sisifofit.com** — Dinámico, moderno y orientado al fitness/entrenamiento.
- **teamsisifo.com** — Ideal si el gimnasio tiene enfoque en entrenamiento grupal o comunidad.
- **sisifoclub.com** — Aporta un perfil más premium o exclusivo.

Opciones .com.ar (Geolocalizadas y profesionales para Argentina)

- **sisifogym.com.ar** — La opción estándar local más sólida.
- **sisifofit.com.ar** — Corto, fácil de deletrear y enfocado al rubro fitness.
- **gymsisifo.com.ar** — Invierte el orden para priorizar la actividad de búsqueda.
- **espaciosisifo.com.ar** — Excelente si combina gimnasio con otras actividades (funcional, nutrición, etc.).
- **sisifotraining.com.ar** — Enfoque más técnico y profesional, ideal para entrenamiento funcional o de fuerza.

Prompt 2: Explicá en términos sencillos y paso a paso el recorrido completo de una petición HTTP GET en la arquitectura cliente-servidor, desde que un usuario ingresa una URL en el navegador hasta que la página se renderiza. Asegurate de detallar:

1. El rol del servidor DNS en la resolución de la IP.
2. Cómo viaja la petición HTTP GET hacia el servidor de hosting.
3. Cómo el servidor procesa y envía la respuesta con los archivos (HTML, CSS, JS, etc.).
4. Qué datos clave se pueden observar en las herramientas de desarrollo del navegador (DevTools a Network) para cada paso.

Respuesta:

Recorrido de una petición HTTP GET: paso a paso

En una arquitectura **cliente-servidor**, el navegador funciona como **cliente** y el servidor donde está alojada la página funciona como **servidor**. El recorrido comienza cuando el usuario escribe una URL y termina cuando el navegador muestra la página.

1. El usuario ingresa la URL

El usuario escribe, por ejemplo:

<https://www.sisifogym.com>

y presiona Enter.

El navegador interpreta la URL y necesita averiguar a **qué servidor debe conectarse**.

2. El DNS encuentra la dirección IP

El navegador consulta al **servidor DNS (Domain Name System)**.

El DNS funciona como una especie de **agenda telefónica de Internet**: transforma un nombre fácil de recordar, como:

`www.sisifogym.com`

en una dirección IP, por ejemplo:

`203.0.113.10` (IP utilizada solo como ejemplo).

El navegador utilizará esa IP para localizar el servidor que aloja la página.

Importante: el DNS no entrega la página web. Su función principal en este proceso es **resolver el nombre de dominio y obtener la IP correspondiente**.

3. El navegador se conecta con el servidor

Una vez conocida la IP, el navegador puede comunicarse con el **servidor de hosting**.

En una conexión HTTPS, además de establecer la comunicación, se realiza la negociación de seguridad mediante **TLS**.

Luego, el navegador envía una petición HTTP utilizando el método **GET**.

Por ejemplo:

GET /index.html HTTP/1.1

Host: www.sisifogym.com

Esto significa, básicamente:

"Servidor, quiero obtener el recurso `/index.html` de este sitio."

La petición viaja desde el dispositivo del usuario, pasando por la red y distintos equipos intermedios, hasta llegar al servidor correspondiente.

4. El servidor recibe y procesa el GET

El servidor web recibe la petición y analiza qué recurso está solicitando el navegador.

Por ejemplo:

GET /index.html

El servidor busca ese archivo dentro del sitio web.

Si el recurso existe y puede ser entregado, prepara una respuesta HTTP.

Por ejemplo:

HTTP/1.1 200 OK

Content-Type: text/html

El código **200 OK** indica que la solicitud se procesó correctamente.

5. El servidor envía los archivos

El servidor devuelve el recurso solicitado al navegador. En este caso, puede enviar el archivo **HTML**.

Cuando el navegador analiza ese HTML, descubre que necesita otros recursos, como:

- CSS → diseño y estilos.
- JavaScript → comportamiento e interacción.

- Imágenes → contenido visual.
- Fuentes → tipografía.
- Otros archivos necesarios.

Entonces realiza **nuevas peticiones GET** para obtenerlos.

Por ejemplo:

GET /index.html

GET /styles.css

GET /script.js

GET /logo.png

El servidor responde a cada solicitud enviando el recurso correspondiente.

6. El navegador procesa los recursos

Una vez recibidos los archivos, el navegador comienza a construir la página.

De forma simplificada:

HTML → estructura de la página

CSS → apariencia y diseño

JS → comportamiento e interacción

Imágenes/fuentes → contenido visual

El navegador interpreta toda esta información y construye lo que finalmente vemos en pantalla.

7. La página se renderiza

Finalmente, el navegador realiza el **renderizado**, es decir, transforma los archivos recibidos en la interfaz visual que el usuario puede observar e interactuar.

El recorrido completo puede resumirse así:

Usuario

|

| Ingresa URL

▼

Navegador (Cliente)

|

| Consulta dominio

▼

DNS

|

| Devuelve IP

▼

Navegador

|

| HTTP GET

▼

Servidor de Hosting

|

| Procesa solicitud

|

| HTTP Response



Navegador

|

| HTML, CSS, JS, imágenes...



Procesamiento y renderizado

|



Página web visible

¿Qué podemos observar en DevTools → Network?

Las herramientas de desarrollo permiten **ver este intercambio entre cliente y servidor**.

Para observarlo, se puede abrir el navegador, presionar **F12** o **Ctrl + Shift + I** y entrar en **Network**. Después se recarga la página.

1. Petición

Al seleccionar un recurso, por ejemplo **index.html**, en **Headers** se pueden observar datos como:

- **Request URL:** dirección del recurso solicitado.
- **Request Method:** **GET**.

- **Status Code:** código recibido del servidor, por ejemplo 200.
- **Host:** servidor al que se realizó la solicitud.
- **Request Headers:** información adicional enviada por el navegador.

Ejemplo:

Request URL: <https://www.sisifogym.com/index.html>

Request Method: GET

Status Code: 200 OK

2. Respuesta

En la misma herramienta también se puede observar información sobre lo que devuelve el servidor:

- **Response Headers:** información enviada por el servidor.
- **Content-Type:** tipo de contenido, por ejemplo [text/html](#).
- **Response:** contenido que recibió el navegador.
- **Size:** tamaño del recurso recibido.
- **Time:** tiempo que tardó la solicitud.

Por ejemplo:

Status Code: 200 OK

Content-Type: text/html

Size: 15.2 kB

Time: 120 ms

3. Recursos adicionales

En **Network** también aparecen las solicitudes de los demás recursos:

Recurso	Método	Estado	Tipo
<code>index.html</code>	GET	200	documento
<code>styles.css</code>	GET	200	stylesheet
<code>script.js</code>	GET	200	script
<code>logo.png</code>	GET	200	image

De esta manera, **Network** permite comprobar prácticamente todo el recorrido HTTP: qué solicita el navegador, qué servidor responde, qué recursos se descargan, cuánto tardan y con qué código de estado responde cada petición.

En resumen: el **DNS** encuentra la IP, el cliente envía una petición **GET** al servidor, el servidor procesa la solicitud y devuelve los recursos, y finalmente el navegador procesa esos recursos y renderiza la página.